To do that, run psql and enter the following query. Supply the constraint name from above. The result will be the constraint definition, showing which table and columns it covers.

sql/constraint_definition_ddl.sql

```
SELECT
  connamespace::regnamespace AS schema,
  conrelid::regclass AS table,
  conname AS constraint,
  pg_get_constraintdef(oid) AS definition,
  FORMAT(
    'ALTER TABLE %I.%I ADD CONSTRAINT %I %s;',
    connamespace::regnamespace,
    conrelid::regclass,
    conname,
    pg_get_constraintdef(oid)
  )
FROM
  pg_constraint
WHERE
  conname IN ('fk_rails_e7560abc33');
```

Copy the value from the rightmost definition column. You'll need a trips_copy table to test this on, similar to the users_copy table above. Run the following statements to create a trips_copy table, and then run the ALTER TABLE statement below to add the Foreign Key Constraint.

```
CREATE TABLE trips_copy (
  LIKE trips INCLUDING ALL EXCLUDING INDEXES
);

ALTER TABLE public.trips_copy
ADD CONSTRAINT fk_rails_e7560abc33 FOREIGN KEY (driver_id)
REFERENCES users(id);
```

You've now seen one way to add the constraint to the destination table. You can DROP TABLE trips_copy; to clean things up.

This is how you'll handle Foreign Key Constraints for columns. Next you'll consider how to handle Sequences used for the Primary Key column. Note that this table is not using an Identity column (See: *Serial type versus Identity columns*[8]) and is using a Sequence for Primary Key column values.

The users table id column is the Primary Key and has a unique value enforced by the Primary Key Constraint, handed out from a Sequence. The Sequence is another database object and has the name users_id_seq which follows the conventional naming format for Sequences from Active Record tables.

8. https://wanago.io/2022/02/21/serial-type-identity-columns-postgresql-typeorm/